In the `com.polarbookshop.orderservice.order.domain` package, create a new `OrderService` class. For starters, let's implement the logic to read orders through the repository. When multiple orders are involved, you can use a `Flux<Order>` object, representing an asynchronous sequence of zero or more orders.

Listing 8.12 Fetching orders via a reactive stream

```
package com.polarbookshop.orderservice.order.domain;

import reactor.core.publisher.Flux;
import org.springframework.stereotype.Service;

@Service                                                    ❶
public class OrderService {
  private final OrderRepository orderRepository;
  public OrderService(OrderRepository orderRepository) {
    this.orderRepository = orderRepository;
  }
  public Flux<Order> getAllOrders() {                       ❷
    return orderRepository.findAll();
  }
}
```

❶ Stereotype annotation that marks a class to be a service managed by Spring

❷ A Flux is used to publish multiple orders (0..N)

Next, we need a method to submit orders. Until we have the integration with Catalog Service in place, we can always default to rejecting submitted orders. `OrderRepository` exposes a `save()` method provided by `ReactiveCrudRepository`. You can build a reactive stream to pass an object of type `Mono<Order>` to `OrderRepository` that will save the order in the database.

Given an ISBN identifying a book and the number of copies to order, you can build a `Mono` object with `Mono.just()` in the same way you